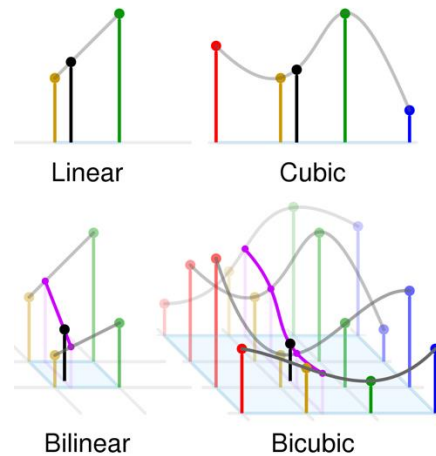


Computational Physics

Lecture 5: Interpolation



Reference: *Computational Physics* by Mark Newman, Chapter 5

<https://websites.umich.edu/~mejn/cp2/chapters/chapter5.pdf>

Chapter 3 of *Numerical Recipes Third Edition* by W.H. Press et al.

Interpolation

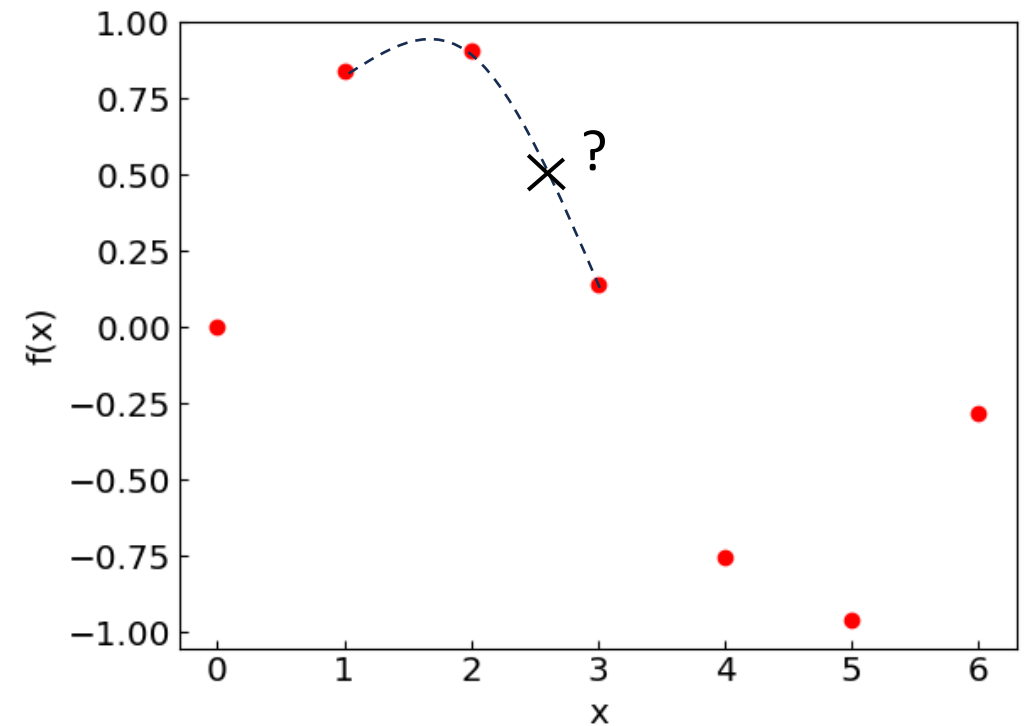
Sometimes we know the value of some function $f(x)$ at a discrete set of points $x_0, x_1, \dots, x_N$, but we do not know how to (easily) calculate its value at an arbitrary x

Interpolation is a mathematical technique used to estimate values between a discrete set of known data points.

It is also employed to approximate an unknown function based on its known values over a certain range.

Two steps:

- 1) Fitting the interpolating function to data points
- 2) Evaluating the interpolating function at a target point x



Linear interpolation

Suppose you have a function $f(x)$ at two points $x = a, b$ and want to estimate a point $x \in [a, b]$

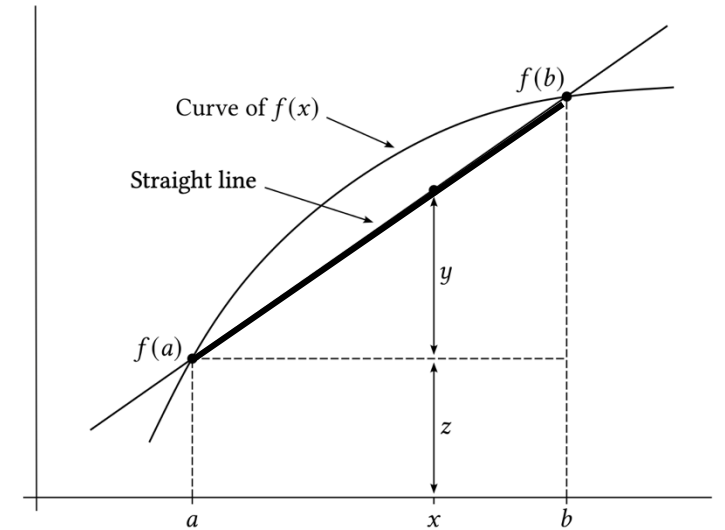
Simplest way is **linear interpolant**: take a **straight line** between these points

Eq. of a straight line with slope and passing in a :

$$f(x) = m(x - a) + f(a) \quad m = \frac{f(b) - f(a)}{b - a}$$

Gives

$$f(x) \simeq \frac{(b - x)f(a) + (x - a)f(b)}{b - a}$$



Can also be used to extrapolate the function to points outside $[a, b]$, though much less accurate

A way to interpret the equation is rewriting in terms of two positive weights: $u_1 = b - x$, $u_2 = x - a$,

$$f(x) \simeq \frac{u_1 f(a) + u_2 f(b)}{u_1 + u_2}.$$

making $f(x)$ as some sort of **weighted average** of the two known values $f(a)$ and $f(b)$ with weights that will depend on the distance of x to a and b

Linear interpolation

Accuracy of linear interpolation: start from
Taylor expansions in a and b

$$\begin{aligned}f(a) &= f(x) + (a - x)f'(x) + \frac{1}{2}(a - x)^2f''(x) + \dots \\f(b) &= f(x) + (b - x)f'(x) + \frac{1}{2}(b - x)^2f''(x) + \dots\end{aligned}$$

Multiply the first of these by $b - x$ and the second by $a - x$, then subtract

$$(b - x)f(a) + (x - a)f(b) = (b - a)f(x) + \frac{1}{2}(a - x)(b - x)(a - b)f''(x) + \dots,$$

Rearranging:

$$f(x) = \underbrace{\frac{(b - x)f(a) + (x - a)f(b)}{b - a}}_{\text{linear interpolation formula}} + \underbrace{(a - x)(b - x)f''(x)}_{\text{leading-order error}} + \dots$$

Error vanishes as x tends to either a or b , and is largest in the middle (assuming $f''(x)$ varies slowly), where it is equal to the central difference:

$$\text{If } h = b - a, \quad \text{at } x = a + \frac{1}{2}h \quad \delta = \frac{1}{4}h^2|f''(x)|$$

Because here we always add we do not need to worry much about rounding errors, typical of subtractions

Lagrange interpolation

With two points only linear interpolation is possible. With more than two points, we can improve in several ways

Lagrange interpolation: interpolate with higher-order polynomials (e.g. quadratic, cubic)

For N points at $x_1 \dots x_N$ in a generic interval $[a,b]$ the unique polynomial of degree $N-1$ that passes exactly in all points $f(x_k)$:

$$\Phi(x) = \sum_{k=1}^N f(x_k) \phi_k(x), \quad \text{with} \quad \phi_k(x) = \prod_{\substack{m=1 \dots N \\ m \neq k}} \frac{(x - x_m)}{(x_k - x_m)} \quad \text{degree } N-1 \text{ polynomials} \quad \phi_k(x_m) = \delta_{km}.$$

You can demonstrate that for all x $\sum_{k=1}^N \phi_k(x) = 1$

$\Phi(x)$ looks now like a weighted average of $f(x)$ values (with $\phi_k(x)$ weights which can be negative)

Lagrange interpolation

Error for the Lagrange polynomial

$$f(x) - \Phi(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

ξ is some point in the interval $[a,b]$

Truncation errors can be a problem if:

- **High-order derivatives** $f^{(n+1)}(x)$ of the function are significant
- The choice of **nodes** leads to a large value of the **product factor**

Other problems might come from the points being noisy: the polynomial will fit exactly all points, producing many wiggles.

Best to fit many lower-order polynomials, which will still give uneven interpolations because the slope of the interpolation changes at the join-points between polynomials.

Examples of interpolating with Lagrange polynomial

Consider the functions: $f(x) = \sin(x)$ with $x \in [0, 2\pi]$

$$f(x) = \frac{1}{1+25x^2} \text{ with } x \in [-1, 1] \text{ (Runge function)}$$

Sample the functions in between the interval using $N = 9, 15$ points.

For each function, construct the Lagrange polynomial that passes through the N points

$$\Phi(x) = \sum_{k=1}^N f(x_k) \phi_k(x), \quad \text{with} \quad \phi_k(x) = \prod_{\substack{m=1 \dots N \\ m \neq k}} \frac{(x - x_m)}{(x_k - x_m)}$$

Tip: use a fine representation of $\Phi(x)$ using e.g. 400 points between the interval. For each point x , you need to compute the value of each $\phi_k(x)$ and the sum $\Phi(x)$.

Plot $f(x)$, $\Phi(x)$ and the points $f(x_k)$ and compare the fitting with different N

Examples of interpolating with Lagrange polynomial

```
def f(x):
    """Runge function"""
    return 1.0 / (1.0 + 25.0 * x**2)

def lagrange_basis(x, nodes, k):
    """phi_k(x) = prod_{m != k} (x - x_m) / (x_k - x_m)."""
    xk = nodes[k]
    phi = np.ones_like(x, dtype=float)
    for m, xm in enumerate(nodes):
        if m == k:
            continue
        phi *= (x - xm) / (xk - xm)
    return phi

def lagrange_interp(x, nodes, values):
    """Phi(x) = sum_k y_k phi_k(x)."""
    Phi = np.zeros_like(x, dtype=float)
    for k in range(len(nodes)):
        Phi += values[k] * lagrange_basis(x, nodes, k)
    return Phi

def uniform_nodes(N, a, b):
    return np.linspace(a, b, N)

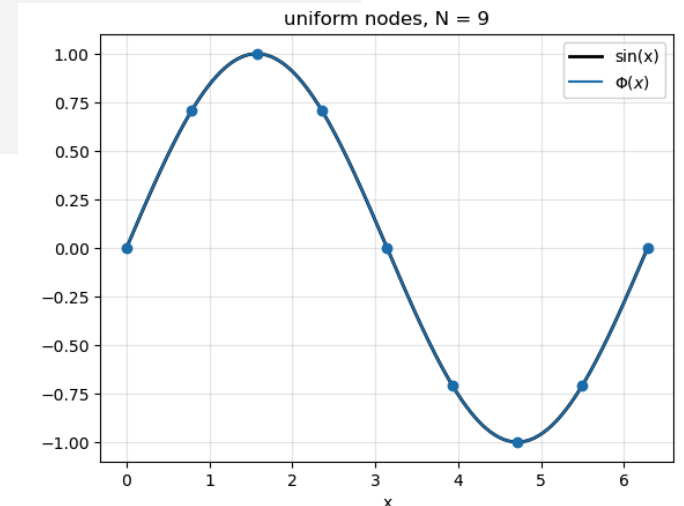
def chebyshev_nodes(N, a, b):
    k = np.arange(1, N + 1)
    s = np.cos((2 * k - 1) * np.pi / (2 * N))[:, :-1]
    return 0.5 * (a + b) + 0.5 * (b - a) * s
```

Interpolating $f(x) = \sin(x)$

```
#interpolation sin with evenly-spaced grid
a=0
b=2*np.pi
x_fine = np.linspace(a, b, 400)
f_fine = np.sin(x_fine)

N=9
nodes = uniform_nodes(N, a, b)
values = np.sin(nodes)
Phi = lagrange_interp(x_fine, nodes, values)

plt.plot(x_fine, f_fine, "k-", lw=2, label="f(x)")
plt.plot(x_fine, Phi, "-", label=fr"$\Phi(x)$")
plt.plot(nodes, values, "o", color="C0")
plt.title(f"uniform nodes, N = {N}")
plt.xlabel("x")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```



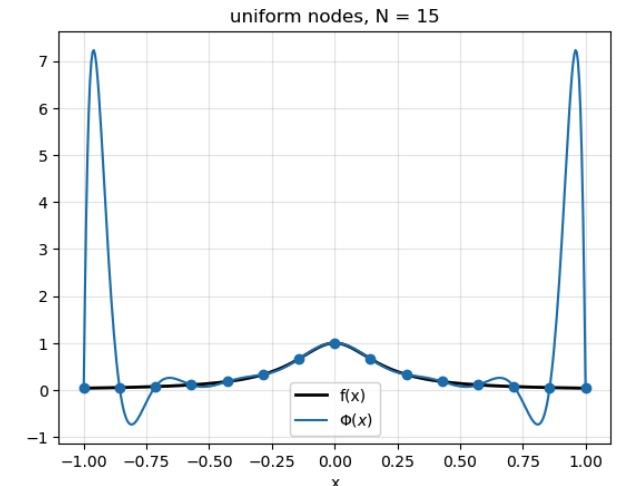
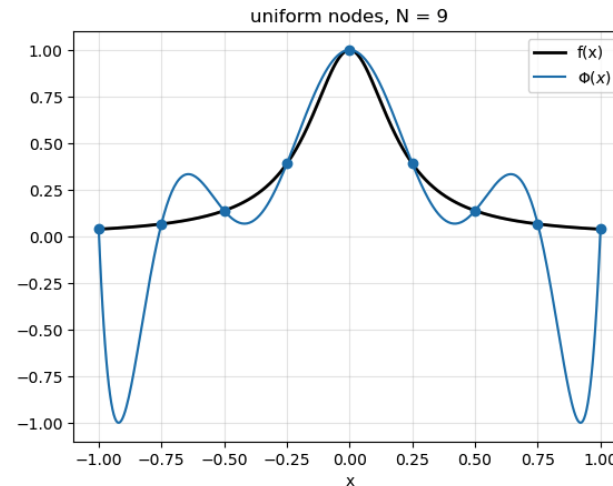
Examples of interpolating with Lagrange polynomial

```
#interpolation Runge function with evenly-spaced grid
a=-1
b=+1
x_fine = np.linspace(a, b, 400)
f_fine = f(x_fine)

N=9
nodes = uniform_nodes(N,a,b)
values = f(nodes)
Phi = lagrange_interp(x_fine, nodes, values)

plt.plot(x_fine, f_fine, "k-", lw=2, label="f(x)")
plt.plot(x_fine, Phi, "-b", label=fr"$\Phi(x)$")
plt.plot(nodes, values, "o", color="C0")
plt.title(fr"uniform nodes, N = {N}")
plt.xlabel("x")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```

$$\text{Interpolating } f(x) = \frac{1}{1+25x^2}$$



Runge phenomenon: oscillations appear at borders.

The error is proportional to the N-derivative, which for this particular function increases fast.

If you try to add some small noise to the points (e.g. a random value taken from a Gaussian) it gets worse.

Examples of interpolating with Lagrange polynomial

Possible mitigation:

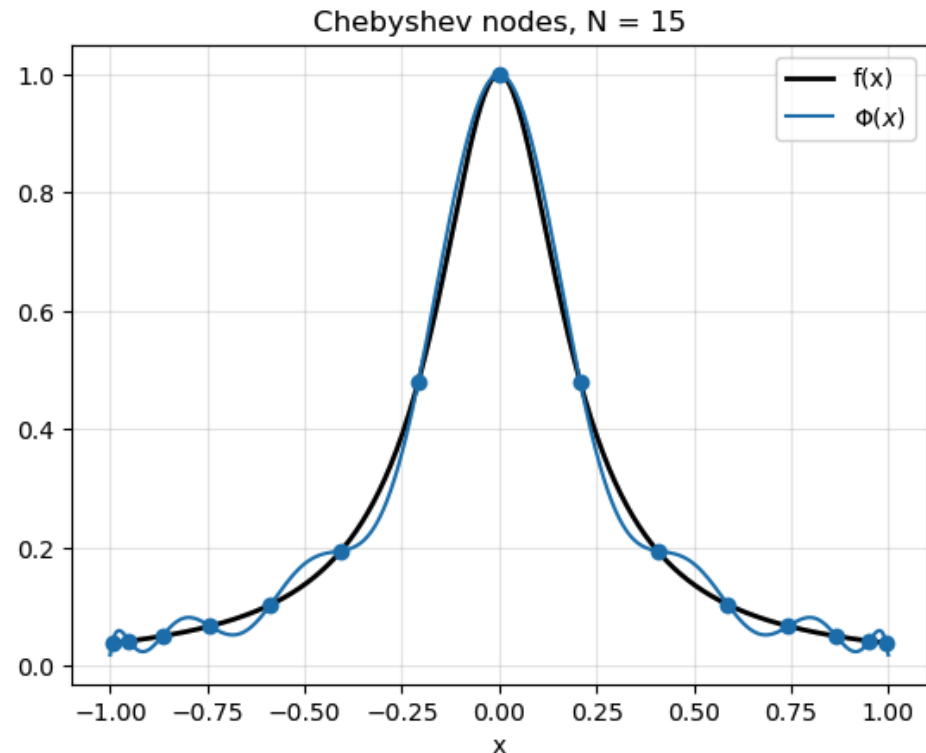
Minimize error by making points denser at edges. Optimal choice are Chebyshev nodes

$$x_k = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{2k+1}{2n+2}\pi\right), \quad k = 0, \dots, n,$$

```
#interpolation Runge function with Chebyshev nodes
a=-1
b=1
x_fine = np.linspace(a, b, 400)
f_fine = f(x_fine)

N=15
nodes = chebyshev_nodes(N,a,b)
values = f(nodes)
Phi = lagrange_interp(x_fine, nodes, values)

plt.plot(x_fine, f_fine, "k-", lw=2, label="f(x)")
plt.plot(x_fine, Phi, "b-", label=fr"$\Phi(x)$")
plt.plot(nodes, values, "o", color="C0")
plt.title(f"uniform nodes, N = {N}")
plt.xlabel("x")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```



Cubic spline interpolation

Another possible mitigation is fitting with smaller degree polynomials, in such a way as to guarantee continuity of function and derivatives: **cubic spline interpolation**.

Connect each pair of nodes x_k by a cubic polynomial

$$q_i(x) = a_i x^3 + b_i x^2 + c_i x + d_i, \quad x \in (x_i, x_{i+1})$$

which needs 4 coefficients a_i, b_i, c_i, d_i determined from:

- data points and continuity
- continuity of 1st and 2nd derivatives
- boundary conditions for first derivative (several choices: e.g. natural means derivative set to zero)

Can give less wiggle and a more natural curvature

Cubic spline interpolation

Using built-in scipy function

```
from scipy.interpolate import CubicSpline

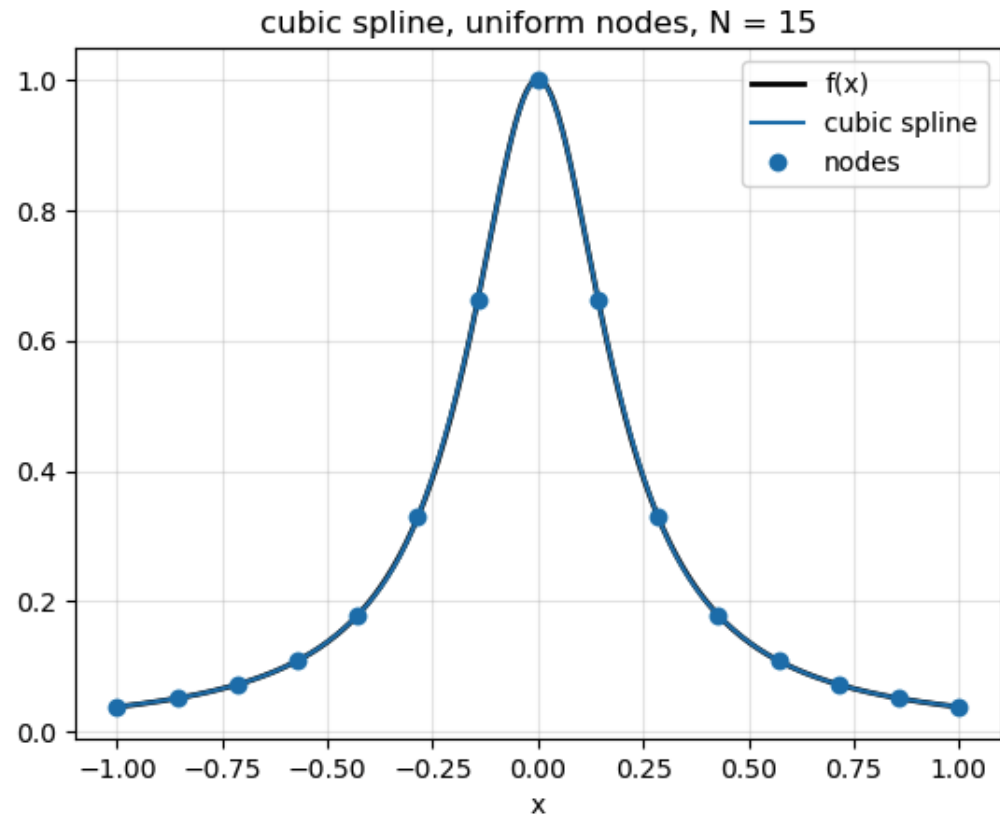
def f(x):
    return 1.0 / (1.0 + 25.0 * x**2)

x_fine = np.linspace(-1.0, 1.0, 400)
f_fine = f(x_fine)

N=15
nodes = np.linspace(-1.0, 1.0, N)
values = f(nodes)

S = CubicSpline(nodes, values, bc_type="natural")

plt.figure()
plt.plot(x_fine, f_fine, "k-", lw=2, label="f(x)")
plt.plot(x_fine, S(x_fine), "-", label="cubic spline")
plt.plot(nodes, values, "o", color="C0", label="nodes")
plt.title(f"cubic spline, uniform nodes, N = {N}")
plt.xlabel("x")
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```



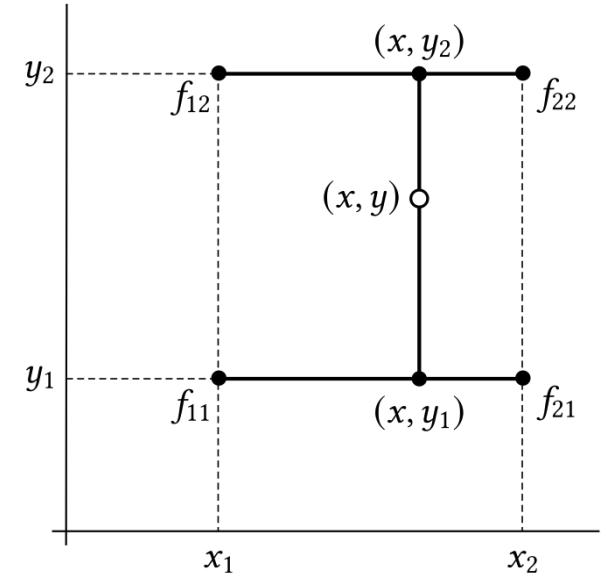
Interpolation in multiple dimensions

Suppose you have a square or rectangular grid of points in 2d measuring some light intensity, and you want to estimate the intensity in between pixels.

We can use **bilinear interpolation**

We start with function $f(x,y)$ for which we know its value on the points of a rectangular grid and take the values of the function at the four corners

$$f_{11} = f(x_1, y_1), \quad f_{21} = f(x_2, y_1), \quad f_{12} = f(x_1, y_2), \quad f_{22} = f(x_2, y_2).$$



To interpolate in a point (x,y) inside the rectangle:

1) interpolate linearly along the two horizontal lines at top and bottom of the rectangle to estimate (x,y_1) and (x,y_2)

$$f(x, y_1) = \frac{(x_2 - x)f_{11} + (x - x_1)f_{21}}{x_2 - x_1},$$

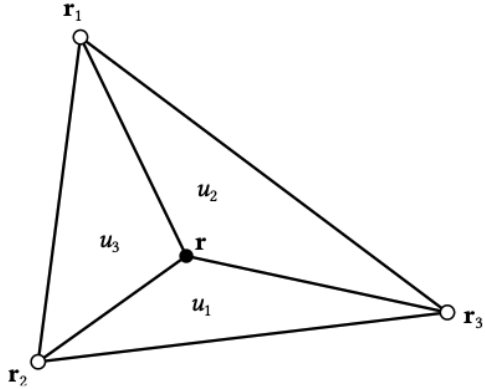
$$f(x, y_2) = \frac{(x_2 - x)f_{12} + (x - x_1)f_{22}}{x_2 - x_1},$$

2) then interpolate vertically between those two points to estimate $f(x,y)$.
$$f(x, y) = \frac{(y_2 - y)f(x, y_1) + (y - y_1)f(x, y_2)}{y_2 - y_1}$$

Same results if we did the opposite starting from the vertical lines

Interpolation: Multiple dimensions

If grid of points is not rectangular, one can interpolate between trios of points forming triangles.



We define the interpolated value of the function $f(\mathbf{r})$

$$f(\mathbf{r}) = \frac{u_1 f_1 + u_2 f_2 + u_3 f_3}{u_1 + u_2 + u_3},$$

where f_1, f_2, f_3 are the values at the corners, and the weights are the area of the three smaller triangles

Area of each triangle can be computed as crossover product between vectors along edge $\frac{1}{2}ab \sin \theta = \frac{1}{2}|\mathbf{a} \times \mathbf{b}|$,

$$u_1 = |(\mathbf{r} - \mathbf{r}_3) \times (\mathbf{r}_2 - \mathbf{r}_3)| = |\mathbf{r} \times (\mathbf{r}_2 - \mathbf{r}_3) + \mathbf{r}_2 \times \mathbf{r}_3|,$$

$$u_2 = |\mathbf{r} \times (\mathbf{r}_3 - \mathbf{r}_1) + \mathbf{r}_3 \times \mathbf{r}_1|,$$

$$u_3 = |\mathbf{r} \times (\mathbf{r}_1 - \mathbf{r}_2) + \mathbf{r}_1 \times \mathbf{r}_2|.$$